

Lab Session : Agenda

1. Print all possible subarrays related to sub-array
2. Sum of odd indexed elements related to PF-sum
3. Special Index related to PF-sum
4. No. of even numbers in given range related to PF-sum
5. Closest Min & Max related to carry-forward

Print Given Subarray

Given an array, start and end index.

Print subarray represented by start and end index.

A : ⁰4 ¹1 ²2 ³3 ⁴-1 ⁵6 ⁶9 S=3 E=5

O/p : 3 -1 6

```
void printSubarray (int A[], int s, int e) {  
    for (int idx = s ; idx ≤ e ; idx++) {  
        print (A[idx])  
    }  
}
```

$$[s \quad e] = e - s + 1$$

↓ ↓

Worst Case 0 N-1 → N

TC : O(N)

SC : O(1)

Print all possible subarrays

$A[] = \overset{0}{4} \quad \overset{1}{13} \quad \overset{2}{8}$

$N = 3$

$$\text{cnt} = \frac{3 \times 4^2}{2} = 6$$

s, e	O/P
0, 0	4
0, 1	4 13
0, 2	4 13 8
1, 1	13
1, 2	13 8
2, 2	8

$$s \leq e$$

Go to every subarray and print it

$N = 3$	$s = 0$	$s = 1$	$s = 2$
$0 \leq i < N$	$e = 0, 1, 2$	$e = 1, 2$	$e = 2$

```
for (s = 0 ; s < N ; s++) <
|
|   for (e = s ; e < N ; e++) <
|   |
|   |   // (s, e)
|   |   print subarray(A, s, e)
```

1 subarray $\rightarrow O(N)$
 N^2 subarray $\rightarrow N^2 \times O(N)$
TC : $O(N^3)$
SC : $O(1)$

Given N array elements & Q queries, for each query calculate sum of all ~~even~~ indexed elements from L to R .

$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5$
 $A[i] = 2 \quad 3 \quad 1 \quad 6 \quad 4 \quad 5$

Queries: $1 \quad 3 \quad A[1] + A[3] = 9$
 $2 \quad 5 \quad A[3] + A[5] = 11$
 $0 \quad 4 \quad A[1] + A[3] = 9$
 $3 \quad 3 \quad A[3] = 6$
 $2 \quad 2 \quad 0$

BF: For each query, iterate from L - R if current idx is odd, add $A[i]$ to sum.

$TC: O(Q \times N)$
 $SC: O(1)$

Optimized: Build $pfsum[]$

$pf[i] = \text{sum of odd indexed elements from } 0 \rightarrow i$

$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5$
 $A[i] = 2 \quad 3 \quad 1 \quad 6 \quad 4 \quad 5$
 $pf[i] = 0 \quad 3 \quad 3 \xrightarrow{+6} 9 \xrightarrow{+4} 13 \xrightarrow{+5} 18$

$i=3 \quad pf[3] = pf[2] + A[3] \quad (i \text{ is odd})$

$i=4 \quad pf[4] = pf[3] \quad (i \text{ is even})$

if $(i/2 == 0) <$

$\mid pf[i] = pf[i-1]$

else <

$\mid pf[i] = pf[i-1] + A[i]$

$A[5] = \begin{matrix} & 0 & 1 & 2 & 3 & 4 \\ 2 & 4 & 3 & 1 & 5 \end{matrix}$
 $pf[5] = 0 \xrightarrow{2} 4 \xrightarrow{3} 3 \xrightarrow{1} 1 \xrightarrow{5} 5$

$A[6] = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \\ 2 & 3 & 1 & 6 & 4 & 5 \end{matrix}$
 $pf[6] = 0 \quad 3 \quad 3 \quad 9 \quad 9 \quad 14$

Queries	L	R	
	1	3	$pf[3] - pf[0] = 9 - 0 = 9$
	2	5	$pf[5] - pf[1] = 14 - 3 = 11$
	0	4	$pf[4] = 9$

$$\text{sum}(L \rightarrow R) = pf[R] - pf[L-1]$$

If $L == 0$

$$\text{sum}(0 \rightarrow R) = pf[R]$$

// A[], N, Query [], Q

```
① int pfo[N]
  pfo[0] = 0
  for (int i = 1; i < N; i++) {
    if (i % 2 == 0) {
      pfo[i] = pfo[i-1]
    }
    else {
      pfo[i] = pfo[i-1] + A[i]
    }
  }
```

} N

```
② for (i = 0; i < Q; i++) { } Q
  int L = Query[i][0]
  int R = Query[i][1]
  int sum = 0
  if (L == 0)
    sum = pfo[R]
  else
    sum = pfo[R] - pfo[L-1]
  print(sum)
```

TC: $O(N + Q)$
SC: $O(N)$
pfo[]

2. Given an array of size N , count number of special index in the array.

NOTE: Special indices are those after removing which, sum of all EVEN indexed elements is equal to sum of all ODD indexed elements.

$A[6] = \langle 4, 3, 2, 7, 6, -2 \rangle$

ans = 2

$S_e == S_o$

Special!

i	0	1	2	3	4	S_e	S_o	Special!
0	3	2	7	6	-2	8	8	✓
1	4	2	7	6	-2	9	8	X
2	4	3	7	6	-2	9	9	✓
3	4	3	2	6	-2	4	9	X
4	4	3	2	7	-2	4	10	X
5	4	3	2	7	6	12	10	X

BF: For each index 'i', check if it is special index $\rightarrow$ create temp[] without $A[i]$.

Iterate to calculate S_e and S_o .

If $(S_e == S_o)$ cnt++

1 idx $\rightarrow 2N$ iterations

N idx $\rightarrow N \times 2N$
 $= 2N^2$

$T_c: O(N^2)$

$S_c: O(N)$

$\downarrow$
temp[]

Quiz 2: Sum of elements at odd indices in resulting array after removal of index 2?

$A[] = \langle 4, 1, 3, 7, 10 \rangle$
 0 1 2 3 4
 4 1 7 10

$S_o = 11$

sum of odd indexed elements after removing idx 2 = sum of ele at odd indices [0-1] + sum of ele at even indices [3-4]

Quiz 3: Sum of elements at odd indices in resulting array after removal of index 3?

$A[] = \langle 2, 3, 1, 4, 0, -1, 2, -2, 10, 8 \rangle$
 0 1 2 3 4 5 6 7 8 9
 2 3 1 0 -1 2 -2 10 8

$S_o = 15$

sum of odd indexed elements after removing idx 3 = sum of ele at odd indices from [0-2] + sum of ele at even indices from [4-9]

Quiz 4: Sum of elements at EVEN indices in resulting array after removal of index 3?

$A[] = \langle \overset{0}{2} \overset{1}{3} \overset{2}{1} \overset{3}{4} \overset{4}{0} \overset{5}{-1} \overset{6}{2} \overset{7}{-2} \overset{8}{10} \overset{9}{8} \rangle$

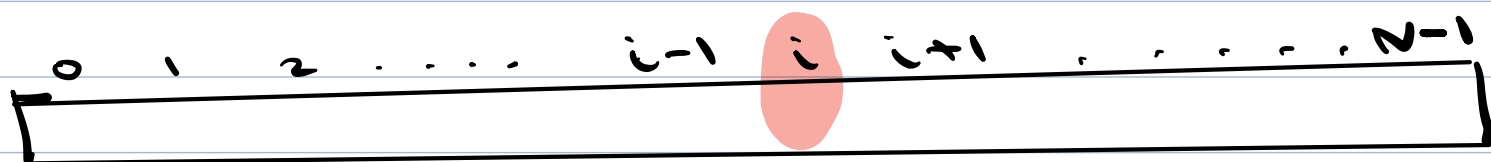
sum of even indexed elements after removing idx 3 = sum of ele at even indices from [0-2] + sum of ele at odd indices [4-9]

$$= (2 + 1) + (-1 + -2 + 8) \\ = 3 + 5 = 8$$

Observations for optimised approach:

1. On deleting i^{th} ele, indices of elements present on left side of i remain intact
2. But indices of elements present on right side

CHANGE $\rightarrow$ element at odd index shift to even and vice versa



$$\text{sum of odd} = S_o [0 \rightarrow i-1] + S_e [i+1 \rightarrow N-1]$$

$$\text{sum of even} = S_e [0 \rightarrow i-1] + S_o [i+1 \rightarrow N-1]$$

$A = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 2 & 3 & 1 & 4 & 0 & -1 & 2 & -2 & 10 & 8 \\ 2 & 3 & 1 & 0 & -1 & 2 & -2 & 10 & 8 \end{matrix}$

sum of odd indexed elements after removing idk 3 = sum of odd indexed elements from 0-2 + sum of even indexed elements from 4-9

sum of even indexed elements after removing idk 3 = sum of even indexed elements from 0-2 + sum of odd indexed from 4-9

-

1. Create $pfe[N]$ and $pfo[N]$
2. Go to every idk i and check where its special or not?
3. Calculate sum_e and sum_o
4. If equal, $cnt++$

int countSpecialIndex (A[], int N) {

// Build pfo[N], pfc[N]

int ans = 0

for (i = 0; i < N; i++) {

// check if ith idx is special?

if (i == 0) {

sum_odd =

sum_even =

pfc[N-1] - pfc[i]

pfo[N-1] - pfo[i]

else {

sum_odd = pfo[i-1] + pfc[N-1] - pfc[i]

sum_even = pfc[i-1] + pfo[N-1] - pfo[i]

if (sum_e == sum_o)

ans++

}

return ans

$N + N + N = 3N$

TC: $O(N)$

SC: $O(N)$

↓
 $2N \rightarrow$ pfo[]
 pfc[]



$$\begin{aligned}\text{sum of odd} &= S_o [0 \rightarrow i-1] + S_e [i+1 \rightarrow N-1] \\ &= pf_o [i-1] + pf_e [N-1] - \\ &\quad pf_e [i+1-1]\end{aligned}$$

$$\text{sum_odd} = pf_o [i-1] + pf_e [N-1] - pf_e [i]$$

$$\begin{aligned}\text{sum of even} &= S_e [0 \rightarrow i-1] + S_o [i+1 \rightarrow N-1] \\ &= pf_e [i-1] + pf_o [N-1] - \\ &\quad pf_o [i+1-1]\end{aligned}$$

$$\text{sum_even} = pf_e [i-1] + pf_o [N-1] - pf_o [i]$$

$$\text{sum}(L-R) = pf[R] - pf[L-1]$$

10:48

3. Given an array of size N and Q queries with L and R . For every query, return count of even elements from L to R .

$A = \langle \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ 2 & 7 & 9 & 3 & 6 & 8 & 5 \end{matrix} \rangle$

$Q = 3$

L	R	cnt
3	6	2
1	2	0
0	5	3

$pf[i] =$ count of even elements from $0-i$

$A = \langle \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ 2 & 7 & 9 & 3 & 6 & 8 & 5 \end{matrix} \rangle$

$pf[i] = \begin{matrix} 1 & 1 & 1 & 1 & 2 & 3 & 3 \end{matrix}$

// $A, N, \text{Query } [L, R], Q$

①

int $pf[N]$

if $(A[0] \% 2)$

$pf[0] = 1$

else $pf[0] = 0$

for $(i = 1; i < N; i++)$

if $(A[i] \% 2 == 0)$

$pf[i] = pf[i-1] + 1$

else

↓
pf [i] = pf [i-1]

② for (i = 0; i < Q; i++) > Q
 int L = Query [i] [0]
 int R = Query [i] [1]
 int sum = 0
 if (L == 0)
 sum = pf [R]
 else
 sum = pf [R] - pf [L-1]
 print (sum)

TC: O(N + Q)

SC: O(N)

↓
pf []

4.

Given an array find length of smallest subarray which contains both min & max of array.

Ex 1 $A = [1^0 \quad 3^1 \quad 2^2]$ min 1 Max 3

s	e	len
0	2	3
0	1	2

ans = 2

Ex 2 0 1 2 3 4 5 6 7 8 9 min max
 1, 2, 3, 1, 3, 4, 6, 4, 6, 3 1 6

s	e	len
0	6	7
3	8	6
0	8	9
3	6	4

ans = 4

Ex 2 0 1 2 3 min max ans = 1
 7 7 7 7 7 7

sol 1

Best 1 Worst N

- ① Find min and max
- ② Consider all subarrays
- ③ Iterate each subarr

// A[], n

① Find min and max

② int ansLength = N ∞

```
for (s = 0; s < N; s++) <
    for (e = s; e < N; e++) <
        // (s, e)
        boolean hasMin = false;
        boolean hasMax = false;
```

```
        for (i = s; i <= e; i++) <
```

```
            if (A[i] == min) <
                hasMin = true
```

```
            if (A[i] == max) <
                hasMax = true
```

```
        if (hasMin == true &&
            hasMax == true) <
```

```
            ans = min(ans, e - s + 1)
```

return ans

1 subarray
→ $O(N)$

 N^2 subarray
→ $N^2 \times N$

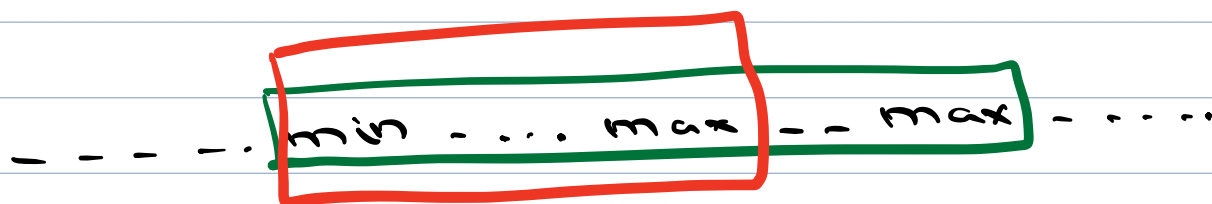
TC : $O(N^3)$

SC : $O(1)$

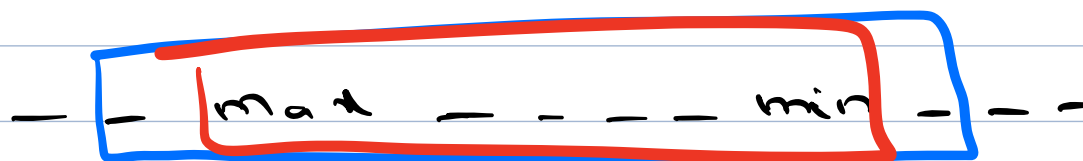
Sol 2 → Build ans subarray directly

Observations

1. Ans subarray will have exactly 1 min and 1 max.



2. Min and max will be placed at corners in ans subarray



3. [min max]
OR
[max min]

2 4 1 2 6 3 6 1

A horizontal sequence of numbers: 2, 4, 1, 2, 6, 3, 6, 1. A green arrow points from the '1' at index 3 to the '6' at index 5.

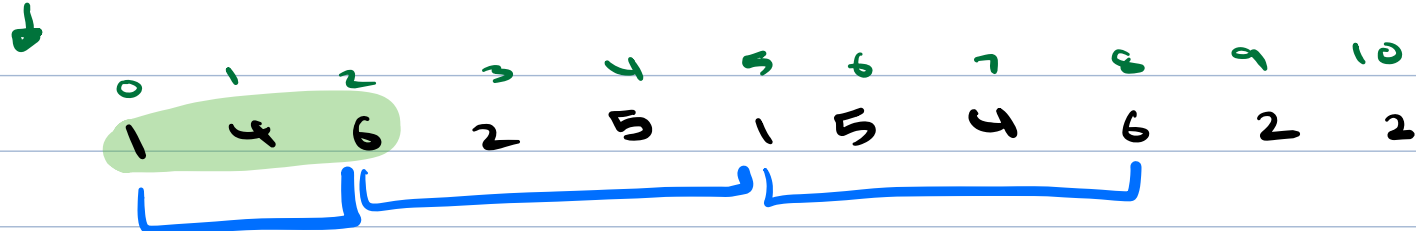
If el is min
closest max on right

If el is max
closest min on right

0 1 2 3 4 5 6 7 8 9 Min Max
 1, 2, 3 1, 3 4, 6, 4, 6, 3 1 6

go to each ele < N²
 if ele is min
 if ele is max
 if ele is closest max on right → loop
 if ele is closest min on right → loop

1. Iterate from right to left
2. Maintain latest-idx-max and latest-idx-min



L-max = ~~2~~

L-min = ~~0~~

ans = ~~2~~ 3

Min
↓
1

Max
↓
6

1. Find max of array $\rightarrow A_{max}$

2. Find min of array $\rightarrow A_{min}$

if ($A_{max} == A_{min}$) return 1

int lastIdxMax = -1, lastIdxMin = -1

int ans = N

for ($i = N - 1$; $i \geq 0$; $i--$) <  

if ($A[i] == A_{min}$) <

// max on right

if ($lastIdxMax != -1$) <

curLength = $lastIdxMax - i + 1$

ans = min(ans, curLength)

lastIdxMin = i

```
else if (ACi == Amax) <
```

```
    // min on right
```

```
    if (lastIdMin != -1) <
```

```
        | curLength = lastIdMin - i + 1
```

```
        | ans = min(ans, curLength)
```

```
        | >
```

```
        lastIdMax = i
```

```
    >
```

```
>
```

```
return ans
```

TC : $O(N)$

SC : $O(1)$